Stream-Sync: A Multithreaded Video Streaming Simulator

A Real-Time Prioritized Consumer-Producer System in C

1. Introduction

This project simulates a real-time video streaming service using multithreaded programming in C. It implements a Producer-Consumer model where the producer thread streams video data in chunks, and multiple consumer threads, each with prioritized access, consume these chunks. The project emphasizes synchronization using POSIX threads, semaphores, and mutexes, along with real-time control via a command interface.

2. Group Members

- Huzaifa Abdul Rehman(23K-0782)
- Ashhad Hasan (23K-0584)
- Umer Siddiqui (23K-0644)
- Sajjad Ahmed (23K-0754)

3. Objectives

- Simulate Concurrent Video Streaming
- Demonstrate Synchronization in C
- Implement Prioritized Consumption
- Enable Real-Time Interaction

4. Tools & Technologies

- Language: C (GCC Compiler)
- Threading: POSIX Threads (pthread.h)
- Synchronization: Mutexes, Semaphores
- Operating System: Linux
- Terminal Control: GNOME Terminal + Named FIFOs
- Performance Logging: Custom logging system using structures and timestamping

5. System Overview

5.1 Architecture

Producer Thread, Consumer Threads (10), and Command Handler Thread form the core components.

5.2 Synchronization Concepts

- Semaphores: Used for consumer signaling and producer control
- Mutexes: Used for buffer access and log protection
- Pause mechanism: Allows pausing/resuming consumers in real-time

6. Use Cases

- Real-Time Prioritized Playback
- Live Pause and Resume
- Performance Logging and Reporting
- Termination Handling

7. Main Flow of the Project

1. Program Initialization (main.c)

- Initializes global structures including the shared buffer, semaphores (sem_empty), and mutexes (mutex, pause_mutex, log_mutex).
- Creates named FIFOs (/tmp/consumer_fifo_X) and launches separate GNOME terminal windows for each consumer.
- Initializes the logging system for performance monitoring.
- Launches the following threads:
    - Producer Thread: Generates data chunks.
    - Consumer Threads: Ten consumers with priority scheduling.
    - Command Handler Thread: Monitors keyboard input for pause/resume/quit commands.

2. Producer Thread Execution (producer.c)

- Iterates 10 times, each time:
    - Waits for an empty slot using sem_wait(sem_empty).
    - Locks the buffer using pthread_mutex_lock(mutex).
    - Inserts a video chunk (e.g., “Chunk 1”) into the buffer.
    - Unlocks the buffer.
    - Signals the highest-priority consumer by posting to its priority_sem.
    - Increments the circular buffer index in.

- After producing all chunks, inserts NULL entries to signal consumers to terminate.

3. Consumer Thread Execution (consumer.c)

- Each consumer opens its respective FIFO and writes output to its GNOME terminal.
- Priority is assigned inversely to consumer ID (lower ID = higher priority).
- Waits on its priority_sem to be signaled by the producer.
- Before processing:
    - Checks if the system is paused using a shared flag.
    - If paused, logs the pause event and sleeps in a loop until resumed.
- Once active:
    - Locks the buffer and reads the chunk at position out.
    - Displays the chunk in its terminal and increments read_count.
    - If all consumers have read the chunk, it is freed and the empty slot is released with sem_post(sem_empty).
    - Unlocks the buffer and logs the processing time.
    - Signals the next consumer in priority order.
- Exits upon encountering a NULL chunk.

4. Command Handler Execution (main.c)

- Runs in a loop in non-blocking terminal input mode.
- Accepts and handles three user commands:
    - 'p': Pauses all consumers by setting a global flag and logs a PAUSE event.
    - 'r': Resumes consumers and logs a RESUME event.
    - 'q': Signals all threads to exit and ends the simulation gracefully.
- Provides real-time control over consumer behavior, simulating network/system interruptions.

5. Thread Joining and Cleanup

- Main thread waits for:
  - Producer to finish producing chunks.
  - All consumer threads to finish processing and terminate.
  - Command handler to detect the 'q' command and exit.

- Once all threads join:
  - Logs and prints final performance statistics, including:
    - Total chunks processed
    - Number of pause/resume events
    - Recent activity logs
  - Cleans up resources:
    - Unlinks all FIFOs.
    - Destroys all mutexes and semaphores to avoid memory leaks

8. Synchronization Mechanisms in Detail

- sem_empty: Controls producer writes
- priority_sem[]: Signals consumers by priority
- mutex: Protects shared variables
- pause_mutex & log_mutex: Pause logic and logging safety

9. File Operations and Data Consistency

FIFO files are used for consumer output display. Buffer is a shared circular structure.

10. Synchronization Guarantees

- Prioritized Access
- Exclusive Modification
- Safe Pausing
- No Deadlocks

11. Program Termination

Producer ends, signals termination to consumers, all threads are joined, logs printed.

12. Demonstration of Utility

- Real-Time Stream SimulationReal-Time Stream Simulation
- Pause/Resume for dynamic controlPause/Resume for dynamic control
- Priority Scheduling in actionPriority Scheduling in action
- Robust Logging SystemRobust Logging System

13. Conclusion

This project offers a robust implementation of multithreading and synchronization concepts by simulating a real-time video stream system. It reflects how OS concepts apply to real-world concurrent systems.

14. Sample Dry Run

Scenario: 3 Consumers, 1 Producer, 1 Pause and Resume Command

1. Producer creates Chunk 1, inserts it into buffer, and signals the highest priority consumer.

2. Consumer 1 (highest priority) reads and processes Chunk 1.

3. Command 'p' is issued, all consumers pause.

4. Command 'r' is issued, consumers resume processing.

5. Consumer 2 and Consumer 3 continue processing subsequent chunks based on priority.

6. After 10 chunks, the producer sends NULL entries, and each consumer exits gracefully.

7. Command 'q' is issued, and the system prints final logs and cleans up.